

自作LR parserを速くする



自己紹介

大阪府内で学生やっています

さっきまで実家（高知県）にいた
荷物がちょっと大きめなのはそのまま来てるため



今日はカロリー高めな
LR parserのコアな話をします！
～夏休み特別編？～

(C++の話をします)

関西Ruby（会議）

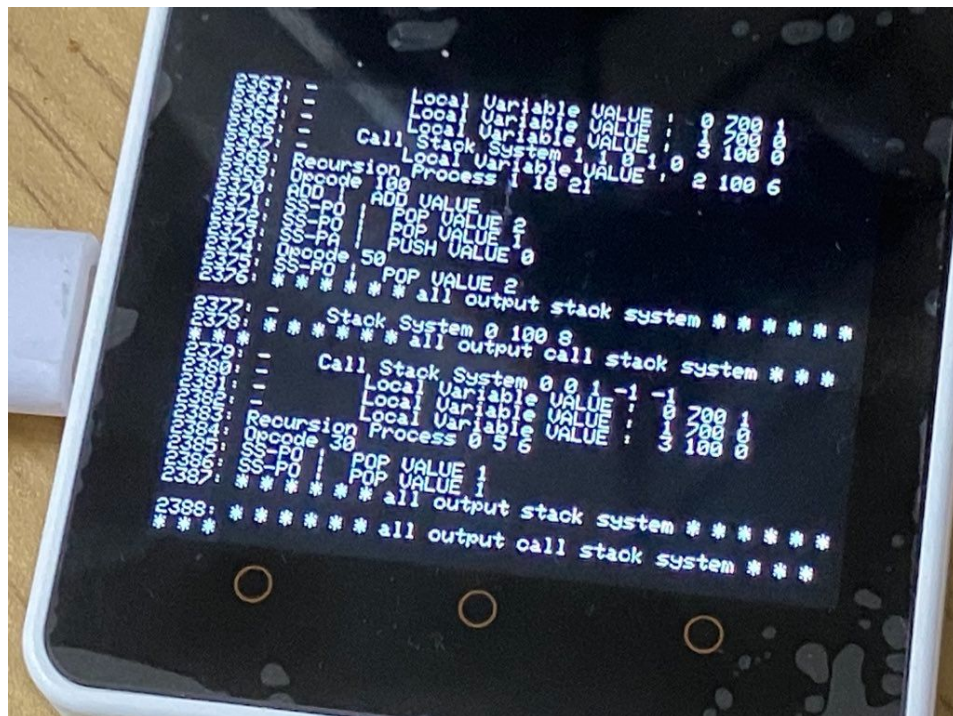
パーサーに向き合う楽しさ

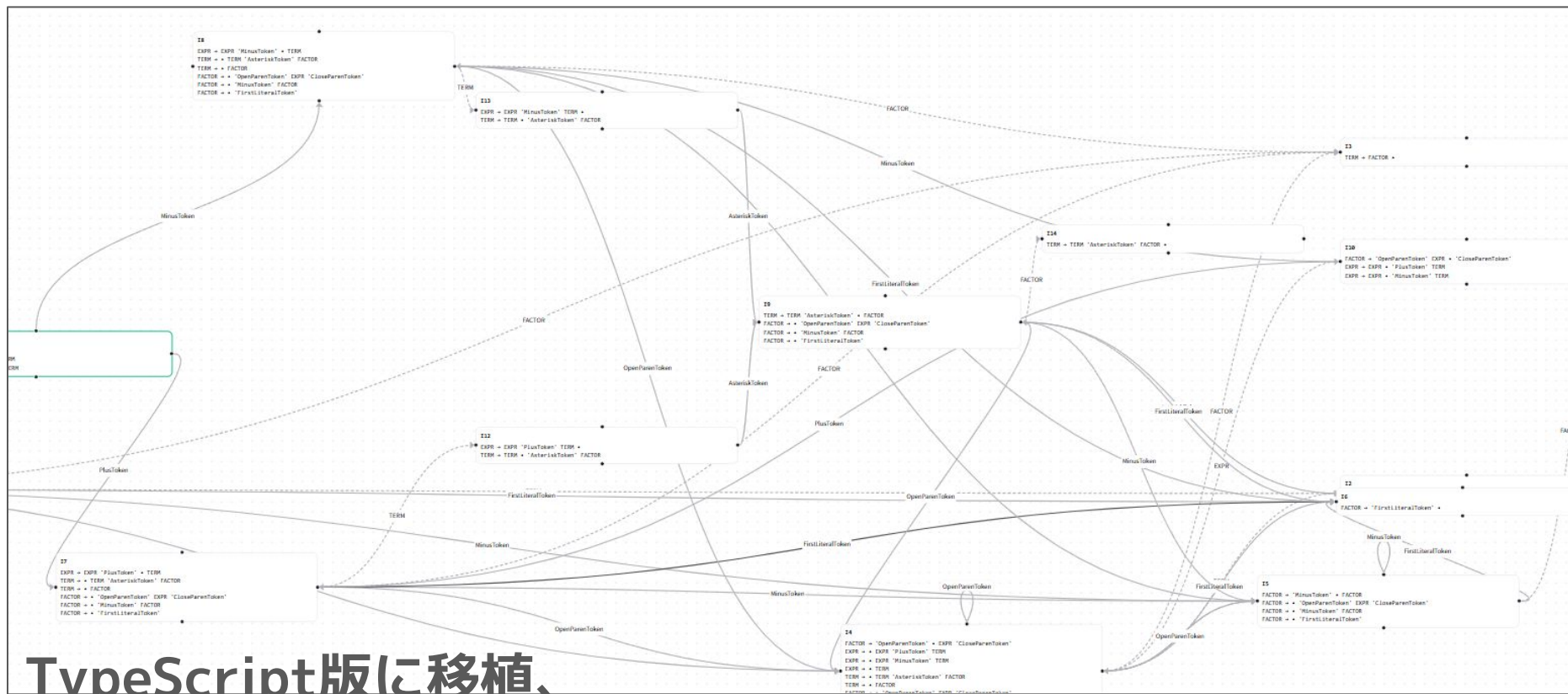
Ruby関西

パーサーに向き合う苦しさ

私にとってLR parserは青春そのもの（？）

いろんな自作言語を支えてきた





TypeScript版に移植、
学生勉強会ではもっぱらこっちが使われるように
(可視化できる機能もあるし・・・)

問題点1

- ・ かなり無理やりな？実装が多い

```
size_t  
size_t ClosureExpansion::getCashKey(vstring null_set, BNFParse::vDeploymentTokenStruct cash_first_set)  
{  
  
    size_t hash = 0;  
    for (const auto &state : cash_first_set)  
    {  
        hash ^= std::hash<string>()(state.token_str) + 0x9e3779b9 + (hash << 6) + (hash >> 2);  
        hash ^= std::hash<int>()(state.label) + 0x9e3779b9 + (hash << 6) + (hash >> 2);  
    }  
}
```

実装における一番の問題点

曲芸的な実装を積み重ねていた

```

namespace ItemSet
void FirstSetClass::recursionFirstsSet(string current_key)
for (int j = 0; j < formula_expansion_vector_size; j++)
if (token_vector_size == 0)
// NULL case
continue;
}
else
{
int k = 0;
int recursion_flag;
do
{
recursion_flag = false;
string token_str = token_vector[k].token_str;
int label = token_vector[k].label;

if (label == is_id_NonterminalSymbolRight)
{
if (current_key == token_str)
{
recursion_flag = true;
}
else
{
recursionFirstsSet(token_str);

for (int fc = 0; fc < first_set[token_str].size(); fc++)
{
BNFParser::DeploymentTokenStruct fc_element = first_set[token_str][fc];
if (!hasDtoken(first_set[current_key], fc_element))
{
first_set[current_key].push_back(fc_element);
}
}
}
}
else if (label == is_id_TerminalSymbol)
{
first_set[current_key].push_back(token_vector[k]);
}
k++;
} while (k < token_vector_size && (hasKeyMap(this->nullable_nonterminals, token_vector[k - 1].token_str) || recursion_flag));
}
}

```

```
} while (k < token_vector_size && (hasKeyMap(this->nullable_nonterminals, token_vector[k - 1].token_str) || recursion_flag));
```

とりあえず動いているからよし . . .

とはいえ、かなりおそい！

DFAノード900個程度にかかる時間

実装当初（3年前）：1時間

第一回高速化：10分

今回、5秒弱まで落とし込むことができた

自作LR parserを高速化しよう！

DFAノードの作成に時間がかかっていた

どうやってLR parserを作ったか？



本を読んでもわからん

(当時学部1年, 生成AIにcoding能力はまだないころ)

LR parserのサンプル実例を眺めて
いい感じになるよう自力でアルゴリズムを考案

- $S \rightarrow \text{EXPR}$
- $\text{EXPR} \rightarrow \text{EXPR PlusToken TERM} \mid \text{EXPR MinusToken TERM} \mid \text{TERM}$
- $\text{TERM} \rightarrow \text{TERM AsteriskToken FACTOR} \mid \text{FACTOR}$
- $\text{FACTOR} \rightarrow \text{OpenParenToken EXPR CloseParenToken} \mid \text{MinusToken FACTOR} \mid \text{FirstLiteralToken}$

↑これを入力したら
↓初期ノードはこうなる

I0

```

S → • EXPR [$]
EXPR → • EXPR 'PlusToken' TERM [$]
EXPR → • EXPR 'MinusToken' TERM [$]
EXPR → • TERM [$]
EXPR → • EXPR 'PlusToken' TERM [PlusToken]
EXPR → • EXPR 'MinusToken' TERM [PlusToken]
EXPR → • TERM [PlusToken]
EXPR → • EXPR 'PlusToken' TERM [MinusToken]
EXPR → • EXPR 'MinusToken' TERM [MinusToken]
EXPR → • TERM [MinusToken]
TERM → • TERM 'AsteriskToken' FACTOR [$]
TERM → • FACTOR [$]

```

- $S \rightarrow \text{EXPR}$
- $\text{EXPR} \rightarrow \text{EXPR PlusToken TERM} \mid \text{EXPR MinusToken TERM} \mid \text{TERM}$
- $\text{TERM} \rightarrow \text{TERM AsteriskToken FACTOR} \mid \text{FACTOR}$
- $\text{FACTOR} \rightarrow \text{OpenParenToken EXPR CloseParenToken} \mid \text{MinusToken FACTOR} \mid \text{FirstLiteralToken}$

↑これを入力したら
↓初期ノードはこうなる

I0

```

S → • EXPR [$]
EXPR → • EXPR 'PlusToken' TERM [$]
EXPR → • EXPR 'MinusToken' TERM [$]
EXPR → • TERM [$]
EXPR → • EXPR 'PlusToken' TERM [PlusToken]
EXPR → • EXPR 'MinusToken' TERM [PlusToken]
EXPR → • TERM [PlusToken]
EXPR → • EXPR 'PlusToken' TERM [MinusToken]
EXPR → • EXPR 'MinusToken' TERM [MinusToken]
EXPR → • TERM [MinusToken]
TERM → • TERM 'AsteriskToken' FACTOR [$]
TERM → • FACTOR [$]

```

辻褄が合うよう
気合いで実装！



I0

$S \rightarrow \bullet \text{EXPR } [\$]$

$\text{EXPR} \rightarrow \bullet \text{EXPR 'PlusToken' TERM } [\$]$

$\text{EXPR} \rightarrow \bullet \text{EXPR 'MinusToken' TERM } [\$]$

$\text{EXPR} \rightarrow \bullet \text{TERM } [\$]$

$\text{EXPR} \rightarrow \bullet \text{EXPR 'PlusToken' TERM } [\text{PlusToken}]$

$\text{EXPR} \rightarrow \bullet \text{EXPR 'MinusToken' TERM } [\text{PlusToken}]$

$\text{EXPR} \rightarrow \bullet \text{TERM } [\text{PlusToken}]$

$\text{EXPR} \rightarrow \bullet \text{EXPR 'PlusToken' TERM } [\text{MinusToken}]$

$\text{EXPR} \rightarrow \bullet \text{EXPR 'MinusToken' TERM } [\text{MinusToken}]$

$\text{EXPR} \rightarrow \bullet \text{TERM } [\text{MinusToken}]$

$\text{TERM} \rightarrow \bullet \text{TERM 'AsteriskToken' FACTOR } [\$]$

$\text{TERM} \rightarrow \bullet \text{FACTOR } [\$]$

I0

S → • EXPR [\$]
EXPR → • EXPR 'PlusToken' TERM [\$]
EXPR → • EXPR 'MinusToken' TERM [\$]
EXPR → • TERM [\$]
EXPR → • EXPR 'PlusToken' TERM [PlusToken]
EXPR → • EXPR 'MinusToken' TERM [PlusToken]
EXPR → • TERM [PlusToken]
EXPR → • EXPR 'PlusToken' TERM [MinusToken]
EXPR → • EXPR 'MinusToken' TERM [MinusToken]
EXPR → • TERM [MinusToken]
TERM → • TERM 'AsteriskToken' FACTOR [\$]
TERM → • FACTOR [\$]

I1 (✓)

S → EXPR • [\$]
EXPR → EXPR • 'PlusToken' TERM [\$]
EXPR → EXPR • 'MinusToken' TERM [\$]
EXPR → EXPR • 'PlusToken' TERM [PlusToken]
EXPR → EXPR • 'MinusToken' TERM [PlusToken]

I2

EXPR → TERM • [\$]
EXPR → TERM • [PlusToken]
EXPR → TERM • [MinusToken]
TERM → TERM • 'AsteriskToken' FACTOR [\$]
TERM → TERM • 'AsteriskToken' FACTOR [PlusToken]

I3

TERM → FACTOR • [\$]
TERM → FACTOR • [PlusToken]
TERM → FACTOR • [MinusToken]
TERM → FACTOR • [AsteriskToken]

I0

S → • EXPR [\$]
EXPR → • EXPR 'PlusToken' TERM [\$]
EXPR → • EXPR 'MinusToken' TERM [\$]
EXPR → • TERM [\$]
EXPR → • EXPR 'PlusToken' TERM [PlusToken]
EXPR → • EXPR 'MinusToken' TERM [PlusToken]
EXPR → • TERM [PlusToken]
EXPR → • EXPR 'PlusToken' TERM [MinusToken]
EXPR → • EXPR 'MinusToken' TERM [MinusToken]
EXPR → • TERM [MinusToken]
TERM → • TERM 'AsteriskToken' FACTOR [PlusToken]
TERM → • FACTOR [PlusToken]

I1 (✓)

S → EXPR • [\$]
EXPR → EXPR • 'PlusToken' TERM [\$]
EXPR → EXPR • 'MinusToken' TERM [\$]
EXPR → EXPR • 'PlusToken' TERM [PlusToken]
EXPR → EXPR • 'MinusToken' TERM [PlusToken]

I2

EXPR → TERM • [\$]
EXPR → TERM • [PlusToken]
EXPR → TERM • [MinusToken]
TERM → TERM • 'AsteriskToken' FACTOR [PlusToken]
TERM → TERM • 'AsteriskToken' FACTOR [PlusToken]

I3

TERM → FACTOR • [\$]
TERM → FACTOR • [PlusToken]
TERM → FACTOR • [MinusToken]
TERM → FACTOR • [AsteriskToken]

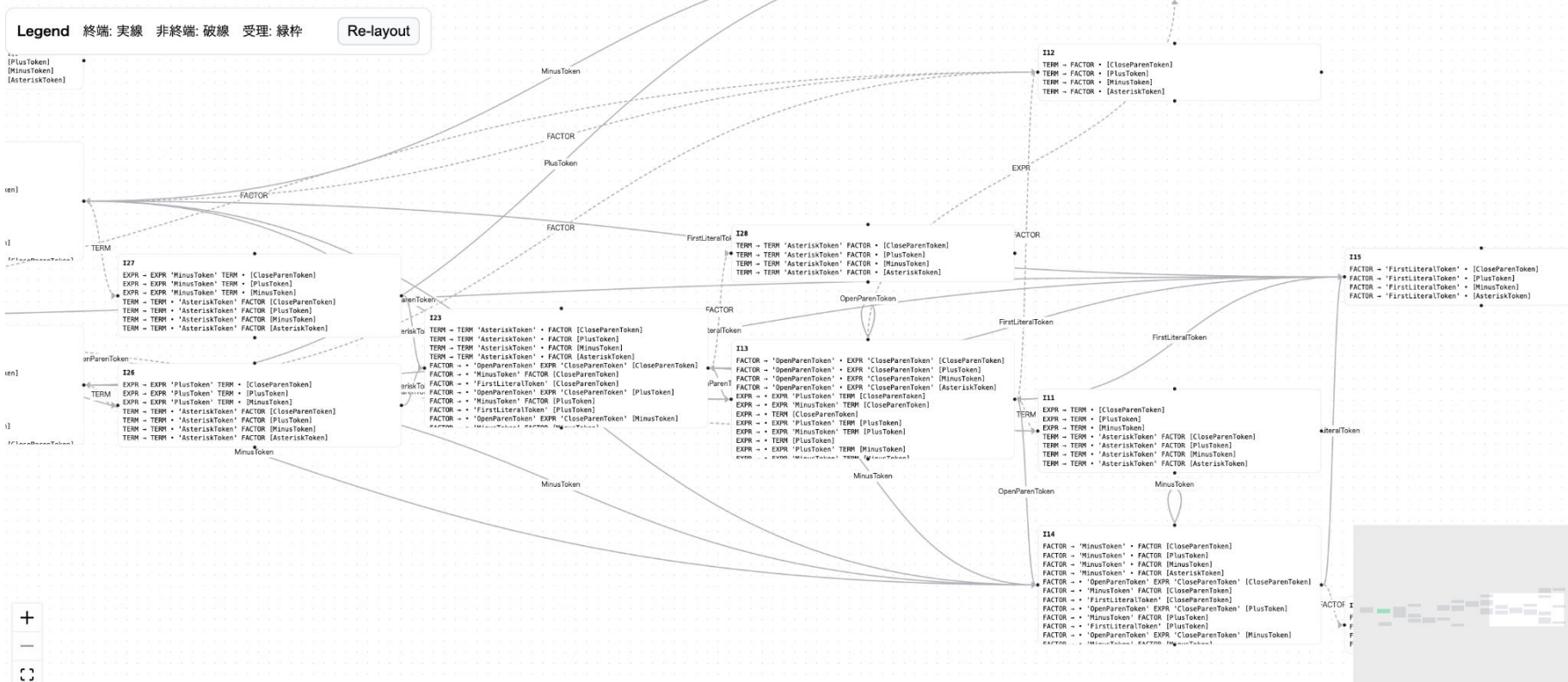
辻褄が合うよう
気合いで実装！



入力と出力は合っているが . . .
非効率

そもそもDFAノードを
データ構造として表現するのが難しい

- ノード数: 30



I8

EXPR → EXPR 'MinusToken' • TERM [\$]

EXPR → EXPR 'MinusToken' • TERM [PlusToken]

EXPR → EXPR 'MinusToken' • TERM [MinusToken]

TERM → • TERM 'AsteriskToken' FACTOR [\$]

TERM → • FACTOR [\$]

TERM → • TERM 'AsteriskToken' FACTOR [PlusToken]

TERM → • FACTOR [PlusToken]

TERM → • TERM 'AsteriskToken' FACTOR [MinusToken]

TERM → • FACTOR [MinusToken]

TERM → • TERM 'AsteriskToken' FACTOR [AsteriskToken]

TERM → • FACTOR [AsteriskToken]

FACTOR → • 'OpenParenToken' EXPR • 'CloseParenToken' [\$]

ノード1つにめちゃくちゃ情報がある
再帰処理がいっぱいある

1つ目の
特にその過程で使う「First集合」の実装が曲者

そもそも「First集合とは」？

I8

EXPR → EXPR 'MinusToken' • TERM [\$]
EXPR → EXPR 'MinusToken' • TERM [PlusToken]
EXPR → EXPR 'MinusToken' • TERM [MinusToken]
TERM → • TERM 'AsteriskToken' FACTOR [\$]
TERM → • FACTOR [\$]
TERM → • TERM 'AsteriskToken' FACTOR [PlusToken]
TERM → • FACTOR [PlusToken]
TERM → • TERM 'AsteriskToken' FACTOR [MinusToken]
TERM → • FACTOR [MinusToken]
TERM → • TERM 'AsteriskToken' FACTOR [AsteriskToken]
TERM → • FACTOR [AsteriskToken]
FACTOR → • 'OpenParenToken' EXPR • 'CloseParenToken' [\$]

先読み記号の導出 []の中身

S → EXPR

EXPR → EXPR PlusToken TERM

| EXPR MinusToken TERM

| TERM

TERM → TERM AsteriskToken FACTOR | FACTOR

FACTOR → OpenParenToken EXPR CloseParenToken

| MinusToken FACTOR | FirstLiteralToken

$S \rightarrow \text{EXPR}$

$\text{EXPR} \rightarrow \text{EXPR PlusToken TERM}$
 $| \text{EXPR MinusToken TERM}$
 $| \text{TERM}$

$\text{TERM} \rightarrow \text{TERM AsteriskToken FACTOR} | \text{FACTOR}$

$\text{FACTOR} \rightarrow \underline{\text{OpenParenToken}} \text{EXPR CloseParenToken}$
 $| \underline{\text{MinusToken}} \text{FACTOR} | \underline{\text{FirstLiteralToken}}$

First(S)だと 下線部の終端記号

$S \rightarrow \text{EXPR}$

$\text{EXPR} \rightarrow \text{EXPR PlusToken TERM} \mid \text{EXPR MinusToken TERM}$
 $\mid \text{TERM}$

$\text{TERM} \rightarrow \text{TERM AsteriskToken FACTOR} \mid \text{FACTOR}$

$\text{FACTOR} \rightarrow \underline{\text{OpenParenToken}} \text{EXPR CloseParenToken}$
 $\mid \underline{\text{MinusToken}} \text{FACTOR} \mid \underline{\text{FirstLiteralToken}}$

First(S)だと 下線部の終端記号

= ある計算式の先頭には 始めかっこ もしくは 数字
閉じかっこは出てこない

都度先読み記号を求める時に計算していた

$\text{First}(S)$, $\text{First}(\text{TERM})$ ごとに . . .

結果は正しい

けど遅い

First(S) = First(EXPR)

First(EXPR) = First(EXPR) $\cup$ First(TERM)

First(TERM) = First(EXPR) $\cup$ First(FACTOR)

**First(FACTOR) = { OpenParenToken , MinusToken ,
FirstLiteralToken }**

メモ化再帰的にできそう！
=> 早くなった

2つ目の難しさ DFAノードの合流判定

同じアイテム集合のノードは合流する

ノードA

TAIL $\rightarrow$ 'ElseKeyword' $\cdot$ 'IfKeyword' 'Identifier' TAIL [\$]

TAIL $\rightarrow$ 'ElseKeyword' $\cdot$ 'Identifier' [\$]

ノードA

TAIL → 'ElseKeyword' • 'IfKeyword' 'Identifier' TAIL [\$]
TAIL → 'ElseKeyword' • 'Identifier' [\$]



ノードB

TAIL → 'ElseKeyword' 'IfKeyword' • 'Identifier' TAIL [\$]

ノードA

TAIL $\rightarrow$ 'ElseKeyword' • 'IfKeyword' 'Identifier' TAIL [\$]

TAIL $\rightarrow$ 'ElseKeyword' • 'Identifier' [\$]



ノードC

TAIL $\rightarrow$ 'ElseKeyword' 'IfKeyword' 'Identifier' • TAIL [\$]

TAIL $\rightarrow$ • 'ElseKeyword' 'IfKeyword' 'Identifier' TAIL [\$]

TAIL $\rightarrow$ • 'ElseKeyword' 'Identifier' [\$]

TAIL $\rightarrow$ • [\$]

ノードB

TAIL → 'ElseKeyword' 'IfKeyword' • 'Identifier' TAIL [\$]



ノードC

TAIL → 'ElseKeyword' 'IfKeyword' 'Identifier' • TAIL [\$]

TAIL → • 'ElseKeyword' 'IfKeyword' 'Identifier' TAIL [\$]

TAIL → • 'ElseKeyword' 'Identifier' [\$]

TAIL → • [\$]

ノードA

TAIL → 'ElseKeyword' • 'IfKeyword' 'Identifier' TAIL [\$]

TAIL → 'ElseKeyword' • 'Identifier' [\$]

ノードB

TAIL → 'ElseKeyword' 'IfKeyword' • 'Identifier' TAIL [\$]

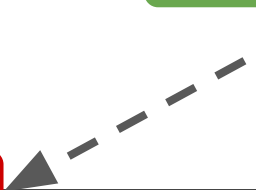
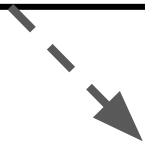
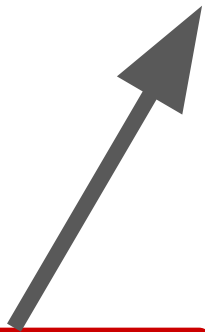
ノードC

TAIL → 'ElseKeyword' 'IfKeyword' 'Identifier' • TAIL [\$]

TAIL → • 'ElseKeyword' 'IfKeyword' 'Identifier' TAIL [\$]

TAIL → • 'ElseKeyword' 'Identifier' [\$]

TAIL → • [\$]



dotとは

LR項

$$X \rightarrow \alpha \cdot Y \beta$$

マーカー(dot)

直感的には、「構文解析の途中経過位置」を表す

この場合、以下の直前の状態を表す

- ・ α までの解析を終了している
- ・ 次に $Y \beta$ の解析を始める

同じアイテム集合なら再利用
これをどう合流判定するか？

```

\Parse
:tDfaSeedHash(DFANode a_node)
{
    int i = 0; i < a_LR_formula_map_keys.size(); i++)
    for (int j = 0; j < a_LR_formula.LR_formula_expansion_vector.size(); j++)
    [
        LRItemFormulaExpansionStruct a_LR_formula_expansion = a_LR_formula.LR_formula_expansion_vector[j];

        BNFParse::vDeploymentTokenStruct a_token_vector = a_LR_formula_expansion.token_vector;
        BNFParse::vDeploymentTokenStruct a_look_ahead = a_LR_formula_expansion.look_ahead;

        int dot = a_LR_formula_expansion.dot;
        int formula_expansion_label = a_LR_formula_expansion.formula_expansion_label;

        hash ^= std::hash<int>()(formula_expansion_label) + 0x9e3779b9 + (hash << 6) + (hash >> 2);
        hash ^= std::hash<int>()(dot) + 0x9e3779b9 + (hash << 6) + (hash >> 2);

        for (int k = 0; k < a_token_vector.size(); k++)
        {
            hash ^= std::hash<int>()(a_token_vector[k].label) + 0x9e3779b9 + (hash << 6) + (hash >> 2);
            hash ^= std::hash<string>()(a_token_vector[k].token_str) + 0x9e3779b9 + (hash << 6) + (hash >> 2);
        }

        for (int k = 0; k < a_look_ahead.size(); k++)
        {
            hash ^= std::hash<int>()(a_look_ahead[k].label) + 0x9e3779b9 + (hash << 6) + (hash >> 2);
            hash ^= std::hash<string>()(a_look_ahead[k].token_str) + 0x9e3779b9 + (hash << 6) + (hash >> 2);
        }
    }
}

```

気合いhash化で $O(1)$ で既存のノードがあるか判定！

結局LR parser実装は最後まで気合いで実装！